JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FileSystemProvider

java.lang.Object
    java.nio.file.spi.FileSystemProvider

```
public abstract class FileSystemProvider
extends Object
```

Service-provider class for file systems. The methods defined by the `Files` class will typically delegate to an instance of this class.

A file system provider is a concrete implementation of this class that implements the abstract methods defined by this class. A provider is identified by a `URI` `scheme`. The default provider is identified by the URI scheme "file". It creates the `FileSystem` that provides access to the file systems accessible to the Java virtual machine. The `FileSystems` class defines how file system providers are located and loaded. The default provider is typically a system-default provider but may be overridden if the system property `java.nio.file.spi.DefaultFileSystemProvider` is set. In that case, the provider has a one argument constructor whose formal parameter type is `FileSystemProvider`. All other providers have a zero argument constructor that initializes the provider.

A provider is a factory for one or more `FileSystem` instances. Each file system is identified by a `URI` where the URI's scheme matches the provider's `scheme`. The default file system, for example, is identified by the URI `"file:///"`. A memory-based file system, for example, may be identified by a URI such as `"memory:///?name=logfs"`. The `newFileSystem` method may be used to create a file system, and the `getFileSystem` method may be used to obtain a reference to an existing file system created by the provider. Where a provider is the factory for a single file system then it is provider dependent if the file system is created when the provider is initialized, or later when the `newFileSystem` method is invoked. In the case of the default provider, the `FileSystem` is created when the provider is initialized.

All of the methods in this class are safe for use by multiple concurrent threads.

Since:

1.7

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	**`FileSystemProvider()`**	Initializes a new instance of this class.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
abstract void	**checkAccess**(Path path, AccessMode... modes)	Checks the existence, and optionally the accessibility, of a file.
abstract void	**copy**(Path source, Path target, CopyOption... options)	Copy a file to a target file.
abstract void	**createDirectory**(Path dir, FileAttribute<?>... attrs)	Creates a new directory.
void	**createLink**(Path link, Path existing)	Creates a new link (directory entry) for an existing file.
void	**createSymbolicLink**(Path link, Path target, FileAttribute<?>... attrs)	Creates a symbolic link to a target.
abstract void	**delete**(Path path)	Deletes a file.
boolean	**deleteIfExists**(Path path)	Deletes a file if it exists.
boolean	**exists**(Path path, LinkOption... options)	Tests whether a file exists.
abstract <V extends FileAttributeView> V	**getFileAttributeView**(Path path, Class<V> type, LinkOption... options)	Returns a file attribute view of a given type.
abstract FileStore	**getFileStore**(Path path)	Returns the FileStore representing the file store where a file is located.
abstract FileSystem	**getFileSystem**(URI uri)	

		Returns an existing `FileSystem` created by this provider.
abstract `Path`	**`getPath`**`(URI uri)`	Return a `Path` object by converting the given `URI`.
abstract `String`	**`getScheme`**`()`	Returns the URI scheme that identifies this provider.
static `List<FileSystemProvider>`	**`installedProviders`**`()`	Returns a list of the installed file system providers.
abstract boolean	**`isHidden`**`(Path path)`	Tests whether or not a file is considered *hidden*.
abstract boolean	**`isSameFile`**`(Path path, Path path2)`	Tests if two paths locate the same file.
abstract void	**`move`**`(Path source, Path target, CopyOption... options)`	Move or rename a file to a target file.
`AsynchronousFileChan`	**`newAsynchronousFileChann`**`(Path path, Set<? extends OpenOption> options, ExecutorService executor FileAttribute<? >... attrs)`	Opens or creates a file for reading and/or writing, returning an asynchronous file channel to access the file.
abstract `SeekableByteChannel`	**`newByteChannel`**`(Path path, Set<? extends OpenOption> options, FileAttribute<? >... attrs)`	Opens or creates a file, returning a seekable byte channel to access the file.
abstract `DirectoryStream<Path>`	**`newDirectoryStream`**`(Path dir, DirectoryStream.Filter<? super Path> filter)`	Opens a directory, returning a `DirectoryStream` to iterate over the entries in the directory.

FileChannel	**newFileChannel**(Path path, Set<? extends OpenOption> options, FileAttribute<?>... attrs)	Opens or creates a file for reading and/or writing, returning a file channel to access the file.
abstract FileSystem	**newFileSystem**(URI uri, Map<String,?> env)	Constructs a new FileSystem object identified by a URI.
FileSystem	**newFileSystem**(Path path, Map<String, ?> env)	Constructs a new FileSystem to access the contents of a file as a file system.
InputStream	**newInputStream**(Path path, OpenOption... options)	Opens a file, returning an input stream to read from the file.
OutputStream	**newOutputStream**(Path path, OpenOption... options)	Opens or creates a file, returning an output stream that may be used to write bytes to the file.
abstract <A extends BasicFileAttributes> A	**readAttributes**(Path path, Class<A> type, LinkOption... options)	Reads a file's attributes as a bulk operation.
abstract Map <String,Object>	**readAttributes**(Path path, String attributes, LinkOption... options)	Reads a set of file attributes as a bulk operation.
<A extends BasicFileAttributes> A	**readAttributesIfExists**(Path path, Class<A> type, LinkOption... options)	Reads a file's attributes as a bulk operation if it exists.
Path	**readSymbolicLink**(Path link)	Reads the target of a symbolic link.
abstract void	**setAttribute**(Path path, String attribute,	Sets the value of a file attribute.

```
                    Object value,
                    LinkOption... options)
```

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

FileSystemProvider

```
protected FileSystemProvider()
```

Initializes a new instance of this class.

During construction a provider may safely access files associated with the default provider but care needs to be taken to avoid circular loading of other installed providers. If circular loading of installed providers is detected then an unspecified error is thrown.

Method Details

installedProviders

```
public static List<FileSystemProvider> installedProviders()
```

Returns a list of the installed file system providers.

The first invocation of this method causes the default provider to be initialized (if not already initialized) and loads any other installed providers as described by the `FileSystems` class.

Returns:

An unmodifiable list of the installed file system providers. The list contains at least one element, that is the default file system provider

Throws:

`ServiceConfigurationError` - When an error occurs while loading a service provider

getScheme

```
public abstract String getScheme()
```

Returns the URI scheme that identifies this provider.

Returns:

The URI scheme

newFileSystem

```
public abstract FileSystem newFileSystem(URI uri,
                                         Map<String,?> env)
                                  throws IOException
```

Constructs a new `FileSystem` object identified by a URI. This method is invoked by the `FileSystems.newFileSystem(URI,Map)` method to open a new file system identified by a URI.

The `uri` parameter is an absolute, hierarchical URI, with a scheme equal (without regard to case) to the scheme supported by this provider. The exact form of the URI is highly provider dependent. The `env` parameter is a map of provider specific properties to configure the file system.

This method throws `FileSystemAlreadyExistsException` if the file system already exists because it was previously created by an invocation of this method. Once a file system is `closed` it is provider-dependent if the provider allows a new file system to be created with the same URI as a file system it previously created.

Parameters:

`uri` - URI reference

`env` - A map of provider specific properties to configure the file system; may be empty

Returns:

A new file system

Throws:

`IllegalArgumentException` - If the pre-conditions for the `uri` parameter aren't met, or the `env` parameter does not contain properties required by the provider, or a property value is invalid

`IOException` - An I/O error occurs creating the file system

`FileSystemAlreadyExistsException` - If the file system has already been created

getFileSystem

```
public abstract FileSystem getFileSystem(URI uri)
```

Returns an existing `FileSystem` created by this provider.

This method returns a reference to a `FileSystem` that was created by invoking the `newFileSystem(URI,Map)` method. File systems created by the `newFileSystem(Path,Map)` method are not returned by this method. The file system is identified by its `URI`. Its exact form is highly provider dependent. In the case of the default provider the URI's path component is `"/"` and the authority, query and fragment components are undefined (Undefined components are represented by `null`).

Once a file system created by this provider is `closed` it is provider-dependent if this method returns a reference to the closed file system or throws `FileSystemNotFoundException`. If the provider allows a new file system to be created with the same URI as a file system it previously created then this method throws the exception if invoked after the file system is closed (and before a new instance is created by the `newFileSystem` method).

Parameters:

`uri` - URI reference

Returns:

The file system

Throws:

`IllegalArgumentException` - If the pre-conditions for the `uri` parameter aren't met

`FileSystemNotFoundException` - If the file system does not exist

getPath

```
public abstract Path getPath(URI uri)
```

Return a `Path` object by converting the given `URI`. The resulting `Path` is associated with a `FileSystem` that already exists or is constructed automatically.

The exact form of the URI is file system provider dependent. In the case of the default provider, the URI scheme is `"file"` and the given URI has a non-empty path component, and undefined query, and fragment components. The resulting `Path` is associated with the default `default` `FileSystem`.

Parameters:

`uri` - The URI to convert

Returns:

The resulting `Path`

Throws:

`IllegalArgumentException` - If the URI scheme does not identify this provider or other preconditions on the uri parameter do not hold

FileSystemNotFoundException - The file system, identified by the URI, does not exist and cannot be created automatically

newFileSystem

```
public FileSystem newFileSystem(Path path,
                                Map<String,?> env)
                         throws IOException
```

Constructs a new FileSystem to access the contents of a file as a file system.

This method is intended for specialized providers of pseudo file systems where the contents of one or more files is treated as a file system. The env parameter is a map of provider specific properties to configure the file system.

If this provider does not support the creation of such file systems or if the provider does not recognize the file type of the given file then it throws UnsupportedOperationException. The default implementation of this method throws UnsupportedOperationException.

Parameters:

path - The path to the file

env - A map of provider specific properties to configure the file system; may be empty

Returns:

A new file system

Throws:

UnsupportedOperationException - If this provider does not support access to the contents as a file system or it does not recognize the file type of the given file

IllegalArgumentException - If the env parameter does not contain properties required by the provider, or a property value is invalid

IOException - If an I/O error occurs

newInputStream

```
public InputStream newInputStream(Path path,
                                  OpenOption... options)
                           throws IOException
```

Opens a file, returning an input stream to read from the file. This method works in exactly the manner specified by the Files.newInputStream(Path, OpenOption...) method.

The default implementation of this method opens a channel to the file as if by invoking the `newByteChannel(Path, Set, FileAttribute...)` method and constructs a stream that reads bytes from the channel. This method should be overridden where appropriate.

Parameters:

`path` - the path to the file to open

`options` - options specifying how the file is opened

Returns:

a new input stream

Throws:

`IllegalArgumentException` - if an invalid combination of options is specified

`UnsupportedOperationException` - if an unsupported option is specified

`IOException` - if an I/O error occurs

newOutputStream

```
public OutputStream newOutputStream(Path path,
                                    OpenOption... options)
                             throws IOException
```

Opens or creates a file, returning an output stream that may be used to write bytes to the file. This method works in exactly the manner specified by the `Files.newOutputStream(Path, OpenOption...)` method.

The default implementation of this method opens a channel to the file as if by invoking the `newByteChannel(Path, Set, FileAttribute...)` method and constructs a stream that writes bytes to the channel. This method should be overridden where appropriate.

Parameters:

`path` - the path to the file to open or create

`options` - options specifying how the file is opened

Returns:

a new output stream

Throws:

`IllegalArgumentException` - if `options` contains an invalid combination of options

`UnsupportedOperationException` - if an unsupported option is specified

`IOException` - if an I/O error occurs

FileAlreadyExistsException - If a file of that name already exists and the CREATE_NEW option is specified *(optional specific exception)*

newFileChannel

```
public FileChannel newFileChannel(Path path,
                                  Set<? extends OpenOption> options,
                                  FileAttribute<?>... attrs)
                           throws IOException
```

Opens or creates a file for reading and/or writing, returning a file channel to access the file. This method works in exactly the manner specified by the FileChannel.open method. A provider that does not support all the features required to construct a file channel throws UnsupportedOperationException. The default provider is required to support the creation of file channels. When not overridden, the default implementation throws UnsupportedOperationException.

Parameters:

path - the path of the file to open or create

options - options specifying how the file is opened

attrs - an optional list of file attributes to set atomically when creating the file

Returns:

a new file channel

Throws:

IllegalArgumentException - If the set contains an invalid combination of options

UnsupportedOperationException - If this provider that does not support creating file channels, or an unsupported open option or file attribute is specified

FileAlreadyExistsException - If a file of that name already exists and the CREATE_NEW option is specified and the file is being opened for writing *(optional specific exception)*

IOException - If an I/O error occurs

newAsynchronousFileChannel

```
public AsynchronousFileChannel newAsynchronousFileChannel
(Path path,
 Set<? extends OpenOption> options,
 ExecutorService executor,
 FileAttribute<?>... attrs)
                                                    throws IOException
```

Opens or creates a file for reading and/or writing, returning an asynchronous file channel to access the file. This method works in exactly the manner specified by the `AsynchronousFileChannel.open` method. A provider that does not support all the features required to construct an asynchronous file channel throws `UnsupportedOperationException`. The default provider is required to support the creation of asynchronous file channels. When not overridden, the default implementation of this method throws `UnsupportedOperationException`.

Parameters:

`path` - the path of the file to open or create

`options` - options specifying how the file is opened

`executor` - the thread pool or `null` to associate the channel with the default thread pool

`attrs` - an optional list of file attributes to set atomically when creating the file

Returns:

a new asynchronous file channel

Throws:

`IllegalArgumentException` - If the set contains an invalid combination of options

`UnsupportedOperationException` - If this provider that does not support creating asynchronous file channels, or an unsupported open option or file attribute is specified

`FileAlreadyExistsException` - If a file of that name already exists and the `CREATE_NEW` option is specified and the file is being opened for writing *(optional specific exception)*

`IOException` - If an I/O error occurs

newByteChannel

```
public abstract SeekableByteChannel newByteChannel
(Path path,
 Set<? extends OpenOption> options,
 FileAttribute<?>... attrs)
                                                throws IOException
```

Opens or creates a file, returning a seekable byte channel to access the file. This method works in exactly the manner specified by the `Files.newByteChannel(Path,Set,FileAttribute[])` method.

Parameters:

`path` - the path to the file to open or create

`options` - options specifying how the file is opened

`attrs` - an optional list of file attributes to set atomically when creating the file

Returns:

a new seekable byte channel

Throws:

`IllegalArgumentException` - if the set contains an invalid combination of options

`UnsupportedOperationException` - if an unsupported open option is specified or the array contains attributes that cannot be set atomically when creating the file

`FileAlreadyExistsException` - If a file of that name already exists and the `CREATE_NEW` option is specified and the file is being opened for writing *(optional specific exception)*

`IOException` - if an I/O error occurs

newDirectoryStream

```
public abstract DirectoryStream<Path> newDirectoryStream
(Path dir,
 DirectoryStream.Filter<? super Path> filter)
                                                throws IOException
```

Opens a directory, returning a `DirectoryStream` to iterate over the entries in the directory. This method works in exactly the manner specified by the `Files.newDirectoryStream(java.nio.file.Path, java.nio.file.DirectoryStream.Filter)` method.

Parameters:

`dir` - the path to the directory

`filter` - the directory stream filter

Returns:

a new and open `DirectoryStream` object

Throws:

`NotDirectoryException` - if the file could not otherwise be opened because it is not a directory *(optional specific exception)*

`IOException` - if an I/O error occurs

createDirectory

```
public abstract void createDirectory(Path dir,
                                     FileAttribute<?>... attrs)
                              throws IOException
```

Creates a new directory. This method works in exactly the manner specified by the `Files.createDirectory(Path, FileAttribute...)` method.

Parameters:

`dir` - the directory to create

`attrs` - an optional list of file attributes to set atomically when creating the directory

Throws:

`UnsupportedOperationException` - if the array contains an attribute that cannot be set atomically when creating the directory

`FileAlreadyExistsException` - if a directory could not otherwise be created because a file of that name already exists *(optional specific exception)*

`IOException` - if an I/O error occurs or the parent directory does not exist

createSymbolicLink

```
public void createSymbolicLink(Path link,
                               Path target,
                               FileAttribute<?>... attrs)
                        throws IOException
```

Creates a symbolic link to a target. This method works in exactly the manner specified by the `Files.createSymbolicLink(Path, Path, FileAttribute...)` method.

The default implementation of this method throws `UnsupportedOperationException`.

Parameters:

`link` - the path of the symbolic link to create

`target` - the target of the symbolic link

`attrs` - the array of attributes to set atomically when creating the symbolic link

Throws:

`UnsupportedOperationException` - if the implementation does not support symbolic links or the array contains an attribute that cannot be set atomically when creating the symbolic link

`FileAlreadyExistsException` - if a file with the name already exists *(optional specific exception)*

`IOException` - if an I/O error occurs

createLink

```
public void createLink(Path link,
                       Path existing)
                throws IOException
```

Creates a new link (directory entry) for an existing file. This method works in exactly the manner specified by the `Files.createLink(Path, Path)` method.

The default implementation of this method throws `UnsupportedOperationException`.

Parameters:

`link` - the link (directory entry) to create

`existing` - a path to an existing file

Throws:

`UnsupportedOperationException` - if the implementation does not support adding an existing file to a directory

`FileAlreadyExistsException` - if the entry could not otherwise be created because a file of that name already exists *(optional specific exception)*

`IOException` - if an I/O error occurs

delete

```
public abstract void delete(Path path)
                    throws IOException
```

Deletes a file. This method works in exactly the manner specified by the `Files.delete(Path)` method.

Parameters:

`path` - the path to the file to delete

Throws:

`NoSuchFileException` - if the file does not exist *(optional specific exception)*

`DirectoryNotEmptyException` - if the file is a directory and could not otherwise be deleted because the directory is not empty *(optional specific exception)*

`IOException` - if an I/O error occurs

deleteIfExists

```
public boolean deleteIfExists(Path path)
                       throws IOException
```

Deletes a file if it exists. This method works in exactly the manner specified by the `Files.deleteIfExists(Path)` method.

The default implementation of this method simply invokes `delete(Path)` ignoring the `NoSuchFileException` when the file does not exist. It may be overridden where appropriate.

Parameters:

`path` - the path to the file to delete

Returns:

`true` if the file was deleted by this method; `false` if the file could not be deleted because it did not exist

Throws:

`DirectoryNotEmptyException` - if the file is a directory and could not otherwise be deleted because the directory is not empty *(optional specific exception)*

`IOException` - if an I/O error occurs

readSymbolicLink

```
public Path readSymbolicLink(Path link)
                      throws IOException
```

Reads the target of a symbolic link. This method works in exactly the manner specified by the `Files.readSymbolicLink(Path)` method.

The default implementation of this method throws `UnsupportedOperationException`.

Parameters:

`link` - the path to the symbolic link

Returns:

The target of the symbolic link

Throws:

`UnsupportedOperationException` - if the implementation does not support symbolic links

`NotLinkException` - if the target could otherwise not be read because the file is not a symbolic link *(optional specific exception)*

`IOException` - if an I/O error occurs

copy

```
public abstract void copy(Path source,
                          Path target,
                          CopyOption... options)
                   throws IOException
```

Copy a file to a target file. This method works in exactly the manner specified by the `Files.copy(Path,Path,CopyOption[])` method except that both the source and target paths must be associated with this provider.

Parameters:

`source` - the path to the file to copy

`target` - the path to the target file

`options` - options specifying how the copy should be done

Throws:

`UnsupportedOperationException` - if the array contains a copy option that is not supported

`FileAlreadyExistsException` - if the target file exists but cannot be replaced because the `REPLACE_EXISTING` option is not specified *(optional specific exception)*

`DirectoryNotEmptyException` - the `REPLACE_EXISTING` option is specified but the file cannot be replaced because it is a non-empty directory *(optional specific exception)*

`IOException` - if an I/O error occurs

move

```
public abstract void move(Path source,
                          Path target,
                          CopyOption... options)
                   throws IOException
```

Move or rename a file to a target file. This method works in exactly the manner specified by the `Files.move(Path, Path, CopyOption...)` method

except that both the source and target paths must be associated with this provider.

Parameters:

`source` - the path to the file to move

`target` - the path to the target file

`options` - options specifying how the move should be done

Throws:

`UnsupportedOperationException` - if the array contains a copy option that is not supported

`FileAlreadyExistsException` - if the target file exists but cannot be replaced because the `REPLACE_EXISTING` option is not specified *(optional specific exception)*

`DirectoryNotEmptyException` - the `REPLACE_EXISTING` option is specified but the file cannot be replaced because it is a non-empty directory *(optional specific exception)*

`AtomicMoveNotSupportedException` - if the options array contains the `ATOMIC_MOVE` option but the file cannot be moved as an atomic file system operation.

`IOException` - if an I/O error occurs

isSameFile

```
public abstract boolean isSameFile(Path path,
                                   Path path2)
                            throws IOException
```

Tests if two paths locate the same file. This method works in exactly the manner specified by the `Files.isSameFile(Path, Path)` method.

Parameters:

`path` - one path to the file

`path2` - the other path

Returns:

`true` if, and only if, the two paths locate the same file

Throws:

`IOException` - if an I/O error occurs

isHidden

```
public abstract boolean isHidden(Path path)
                          throws IOException
```

Tells whether or not a file is considered *hidden*. This method works in exactly the manner specified by the `Files.isHidden(Path)` method.

This method is invoked by the `isHidden` method.

Parameters:

`path` - the path to the file to test

Returns:

`true` if the file is considered hidden

Throws:

`IOException` - if an I/O error occurs

getFileStore

```
public abstract FileStore getFileStore(Path path)
                                 throws IOException
```

Returns the `FileStore` representing the file store where a file is located. This method works in exactly the manner specified by the `Files.getFileStore(Path)` method.

Parameters:

`path` - the path to the file

Returns:

the file store where the file is stored

Throws:

`IOException` - if an I/O error occurs

checkAccess

```
public abstract void checkAccess(Path path,
                                 AccessMode... modes)
                          throws IOException
```

Checks the existence, and optionally the accessibility, of a file.

This method may be used by the `isReadable`, `isWritable` and `isExecutable` methods to check the accessibility of a file.

This method checks the existence of a file and that this Java virtual machine has appropriate privileges that would allow it access the file according to all of access modes specified in the `modes` parameter as follows:

Value	Description
`READ`	Checks that the file exists and that the Java virtual machine has permission to read the file.

Value	Description
WRITE	Checks that the file exists and that the Java virtual machine has permission to write to the file,
EXECUTE	Checks that the file exists and that the Java virtual machine has permission to execute the file. The semantics may differ when checking access to a directory. For example, on UNIX systems, checking for EXECUTE access checks that the Java virtual machine has permission to search the directory in order to access file or subdirectories.

If the modes parameter is of length zero, then the existence of the file is checked.

This method follows symbolic links if the file referenced by this object is a symbolic link. Depending on the implementation, this method may require to read file permissions, access control lists, or other file attributes in order to check the effective access to the file. To determine the effective access to a file may require access to several attributes and so in some implementations this method may not be atomic with respect to other file system operations.

Parameters:

path - the path to the file to check

modes - The access modes to check; may have zero elements

Throws:

UnsupportedOperationException - an implementation is required to support checking for READ, WRITE, and EXECUTE access. This exception is specified to allow for the Access enum to be extended in future releases.

NoSuchFileException - if a file does not exist *(optional specific exception)*

AccessDeniedException - the requested access would be denied or the access cannot be determined because the Java virtual machine has insufficient privileges or other reasons. *(optional specific exception)*

IOException - if an I/O error occurs

getFileAttributeView

```
public abstract <V extends FileAttributeView> V getFileAttributeView
(Path path,
 Class<V> type,
 LinkOption... options)
```

Returns a file attribute view of a given type. This method works in exactly the manner specified by the Files.getFileAttributeView(Path, Class, LinkOption...) method.

Type Parameters:

`V` - The `FileAttributeView` type

Parameters:

`path` - the path to the file

`type` - the `Class` object corresponding to the file attribute view

`options` - options indicating how symbolic links are handled

Returns:

a file attribute view of the specified type, or `null` if the attribute view type is not available

readAttributes

```
public abstract <A extends BasicFileAttributes> A readAttributes
(Path path,
 Class<A> type,
 LinkOption... options)
                                                                throws
IOException
```

Reads a file's attributes as a bulk operation. This method works in exactly the manner specified by the `Files.readAttributes(Path,Class,LinkOption[])` method.

Type Parameters:

`A` - The `BasicFileAttributes` type

Parameters:

`path` - the path to the file

`type` - the `Class` of the file attributes required to read

`options` - options indicating how symbolic links are handled

Returns:

the file attributes

Throws:

`UnsupportedOperationException` - if an attributes of the given type are not supported

`IOException` - if an I/O error occurs

readAttributes

```
public abstract Map<String,Object> readAttributes
(Path path,
 String attributes,
 LinkOption... options)
                                                 throws IOException
```

Reads a set of file attributes as a bulk operation. This method works in exactly the manner specified by the `Files.readAttributes(Path,String,LinkOption[])` method.

Parameters:

`path` - the path to the file

`attributes` - the attributes to read

`options` - options indicating how symbolic links are handled

Returns:

a map of the attributes returned; may be empty. The map's keys are the attribute names, its values are the attribute values

Throws:

`UnsupportedOperationException` - if the attribute view is not available

`IllegalArgumentException` - if no attributes are specified or an unrecognized attributes is specified

`IOException` - If an I/O error occurs

setAttribute

```
public abstract void setAttribute(Path path,
                                  String attribute,
                                  Object value,
                                  LinkOption... options)
                           throws IOException
```

Sets the value of a file attribute. This method works in exactly the manner specified by the `Files.setAttribute(Path, String, Object, LinkOption...)` method.

Parameters:

`path` - the path to the file

`attribute` - the attribute to set

`value` - the attribute value

`options` - options indicating how symbolic links are handled

Throws:

`UnsupportedOperationException` - if the attribute view is not available

`IllegalArgumentException` - if the attribute name is not specified, or is not recognized, or the attribute value is of the correct type but has an inappropriate value

`ClassCastException` - If the attribute value is not of the expected type or is a collection containing elements that are not of the expected type

`IOException` - If an I/O error occurs

exists

```
public boolean exists(Path path,
                      LinkOption... options)
```

Tests whether a file exists. This method works in exactly the manner specified by the `Files.exists(Path, LinkOption...)` method.

Implementation Requirements:

The default implementation of this method invokes the `checkAccess(Path, AccessMode...)` method when symbolic links are followed. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed and the method `readAttributes(Path, Class, LinkOption...)` is called to determine whether a file exists.

Parameters:

`path` - the path to the file to test

`options` - options indicating how symbolic links are handled

Returns:

`true` if the file exists; `false` if the file does not exist or its existence cannot be determined.

Since:

20

readAttributesIfExists

```
public <A extends BasicFileAttributes> A readAttributesIfExists
(Path path,
 Class<A> type,
 LinkOption... options)
                                                          throws
IOException
```

Reads a file's attributes as a bulk operation if it exists.

The `type` parameter is the type of the attributes required and this method returns an instance of that type if supported. All implementations support a

basic set of file attributes and so invoking this method with a `type` parameter of `BasicFileAttributes.class` will not throw `UnsupportedOperationException`.

The `options` array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed.

It is implementation specific if all file attributes are read as an atomic operation with respect to other file system operations.

Implementation Requirements:

The default implementation of this method invokes the `readAttributes(Path, Class, LinkOption...)` method to read the file's attributes.

Type Parameters:

`A` - The `BasicFileAttributes` type

Parameters:

`path` - the path to the file

`type` - the `Class` of the file attributes required to read

`options` - options indicating how symbolic links are handled

Returns:

the file attributes or null if the file does not exist

Throws:

`UnsupportedOperationException` - if an attributes of the given type are not supported

`IOException` - if an I/O error occurs

Since:

20

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.